

Comparison-Free Bit-Stream Generation for Cost-Efficient Unary Computing

Faeze S. Banitaba*, Amir Hossein Jalilvand*, M. Hassan Najafi†, Sercan Aygun*

*University of Louisiana at Lafayette, †Case Western Reserve University

{faeze.banitaba, amir.jalilvand, sercan.aygun}@louisiana.edu, †mhassan.najafi@case.edu

Abstract—Today, unconventional hardware design techniques based on simple data representations are receiving more and more attention. Unary computing is one of these techniques that processes data in the form of uniform bit-streams. The simplicity of implementing complex arithmetic operations and high tolerance to noise are the crucial advantages of unary systems. However, converting data from weighed binary radix to unary representation with existing *comparator-based* unary number generators is expensive regarding footprint area and power consumption. The problem aggregates when the number of inputs and data precision increase. This work proposes a low-cost, comparison-free, unary number generation mechanism for efficient data conversion from binary radix to unary representation. We introduce a serial and two parallel (an exact and an approximate) unary number generators. Synthesis results show that the proposed method reduces the hardware area, power consumption, and area-delay product for both serial and parallel designs compared to the state-of-the-art converter. We evaluate the efficiency of the proposed converter in four use cases.

I. INTRODUCTION

Designers encounter stringent design constraints in modern digital hardware systems, particularly regarding area footprint and power consumption [1]. In digital embedded systems, the primary considerations are optimizing power consumption and minimizing the physical area, as these factors critically affect the overall cost and energy efficiency of the systems. Traditional digital systems predominantly utilize weighted binary radix-2 for data representation. Computations based on this positional representation are complex and costly to implement [2]. Due to the complexity and high cost associated with conventional binary designs, unconventional design techniques that leverage simpler data representations are gaining increasing attention [3].

Unary computing (UC) [4] is an unconventional technique that processes data in the form of uniform bit-streams with the ability to produce deterministic and accurate results [5], [6]. Input values are encoded in the unary domain through a sequence of one value ('0' or '1') followed by a sequence of the other value ('1' or '0', respectively). For example, 11000 is a left-aligned unary bit-stream representing 2/5. Depending on the location of '1's in the stream, unary bit-streams can be either right- or left-aligned. The value of a unary bit-stream is determined by calculating the ratio of the number of 1s to the total length of the bit-stream, indicating the density or probability of observing 1. While data is typically confined to the [0, 1] interval, values outside this range can be presented by scaling. A key difference from binary radix representation is that all digits in a unary bit-stream have equal weight. This provides immunity to noise, as any bit flip results in only a least significant bit (LSB) error in the represented value.

The primary advantage of UC is its simplicity in implementing complex arithmetic operations, which enables ultra-low-cost hardware designs [5]. For instance, multiplication operation can be realized by bit-wise AND operation on *uncorrelated* unary bit-streams [7]. Moreover, minimum (Min), maximum (Max), and absolute difference

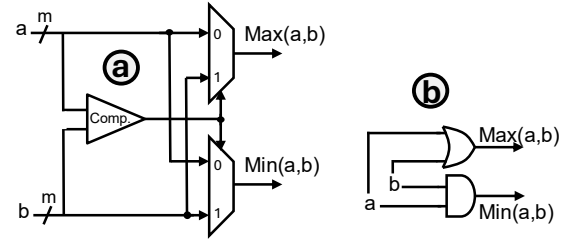


Fig. 1. Implementation of the Min and Max value functions; (a) Conventional binary design (the design cost depends on the data size or m), (b) Unary design (the design cost is independent of the bit-stream length).

operations can be implemented by bit-wise AND, OR, and XOR operations, respectively, on *correlated* unary bit-streams [5]. Fig. 1(a) shows the logic design of the Max and Min value functions in the weighted binary and unary domains. As can be seen, the complexity and implementation cost of the binary design depends on the data precision (m) requiring an m -bit binary comparator and two m -bit multiplexers (MUXs). This results in a higher hardware area and power cost than the unary counterpart implemented using only one AND and one OR gate (Fig. 1(b)). The unary implementation is independent of the data precision. The same design can process data with higher precision by processing longer bit-streams. UC has been used for the hardware-efficient implementation of different applications, from fuzzy logic [8] to hyperdimensional computing [9], image processing [10], deep neural network architecture [11], sorting [12], and neuromorphic computing [13], [14].

An essential step for processing data in unary systems is to convert data from conventional binary to unary bit-streams. A unary bit-stream of 2^m bits is required to accurately represent a data value with a resolution of 2^{-m} [5]. This means a unary bit-stream of 256 bits is needed to accurately represent an 8-bit precision number. Fig. 2(a) illustrates the common structure of a unary bit-stream generator built from an m -bit counter, an m -bit binary comparator, and an m -bit register. The output of comparing the counter and the register values produces one bit of the bit-stream in each clock cycle. A parallel unary bit-stream generator (with degree 2 of parallelism) is also shown in Fig. 2(b). In each clock cycle, the counter output is added by 1 bit ('0' or '1') and compared with the register. Two bits of the unary bit-stream are generated in each cycle with this parallel generator. This structure is costly compared to the computation logic of the unary circuits, consuming over 80% of the unary system's total area and power cost [5].

Stochastic computing (SC) [15] is another emerging computing paradigm that shares many properties with UC but is known for approximate computations and operation on *random* bit-streams. SC uses binary random number generators instead of binary counters for

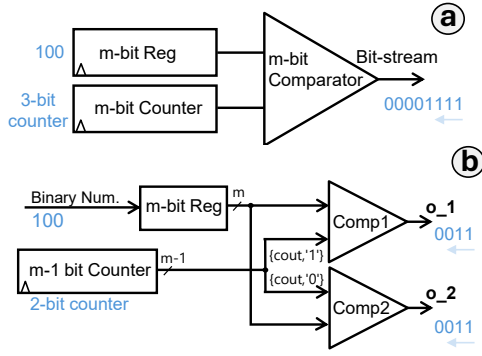


Fig. 2. Conventional comparator-based unary bit-stream generator: (a) A serial design [5], (b) A $2 \times$ parallel design.

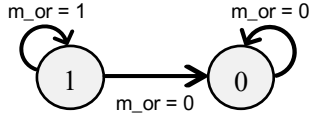


Fig. 3. FSM of the proposed bit-stream generator for left-aligned unary bit-streams, with '1's preceding '0's.

bit-stream generation. While early stochastic bit-stream generators were built using a similar comparator-based structure, finite-state-machine (FSM)-based bit-stream generators have been introduced to reduce the data conversion cost of SC systems. Asadi *et al.* [16] proposed a low-cost FSM-based low-discrepancy (LD) bit-stream generator that supports the generation of quasi-random bit-streams with recurrent LD distributions. Their proposed generator relies on an FSM and a MUX unit for low-cost bit-stream generation. However, their method only supports random bit-streams and cannot generate unary bit-streams.

This work proposes a low-cost, comparison-free unary bit-stream generator that can efficiently convert data from binary radix to unary representation. Synthesis results show up to 85% and 92% savings in the area and power consumption, respectively, compared to those of the conventional comparator-based unary bit-stream generator when converting a single input. When converting eight binary inputs to their corresponding unary bit-streams, up to 62% and 58% savings in the area and power are obtained. But processing fully serial unary bit-streams can result in extended processing latency and high energy consumption. To address this issue, we propose two parallel unary bit-stream generators: 1) an approximate and 2) an accurate parallel bit-stream generator. The accurate design generates unary bit-streams accurately (zero conversion error) at some additional costs compared to the approximate one.

The rest of the paper is organized as follows: Section II describes the proposed unary bit-stream generators. Section III compares the hardware costs of the proposed bit-stream generator to those of the conventional comparator-based generator. Section IV evaluates the proposed bit-stream generator in several use cases. Finally, Section V concludes the paper.

II. PROPOSED ARCHITECTURE

This section discusses the architecture of the proposed bit-stream generator. The proposed generator converts an m -bit precision binary input data to a right-aligned unary bit-stream. The proposed converter is based on a two-state FSM, illustrated in Fig. 3. As can be seen,

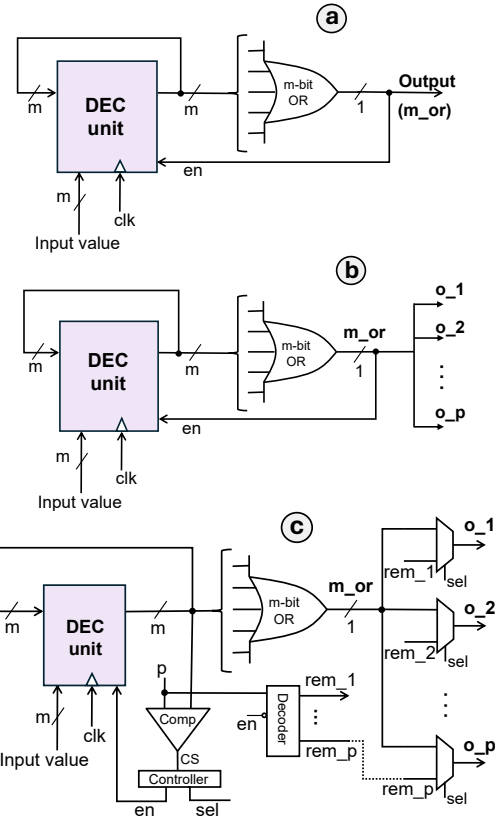


Fig. 4. Proposed designs for unary bit-stream generation: (a) Serial architecture, (b) Approximate Parallel design, (c) Accurate Parallel design.

the FSM has only one transition: when the FSM is in state '1', the output is '1', which then becomes '0' when the state changes to '0'. When the m_or signal becomes '0', the FSM goes from state '1' to state '0'. In what follows, we discuss a serial and two parallel architectures for the proposed bit-stream generator.

A. Serial Design

Fig. 4(a) shows the serial architecture of the proposed bit-stream generator. The core of this bit-stream generator is a decrement (DEC) unit and an m -input OR gate. Consider an input value of 0.5, equivalent to '0.100' in the binary domain with a data precision of $m = 3$. The bit-stream generator converts this binary input to an 8-bit unary bit-stream, i.e., '00001111'. Table I shows a step-by-step example of bit-stream generation with this design. In the proposed design, an input register is initialized by the input value (here, '0.100'). In the first cycle, the enable input of the DEC unit is '0', so the input is not decremented and passes as is. The input enters the m -bit OR gate and re-enters the DEC unit again in the next clock cycle. Now, the output of the OR gate becomes '1'. The same input goes to the DEC unit in the second cycle, but the enable signal is '1' this time. Now, the input is subtracted by one unit and then entered the OR gate. The process continues until the result from the OR gate equals 0, which, as a result, disables the DEC unit. In this state, $m_or=0$, the FSM state changes from '1' to '0', and the output remains on '0' for the remaining cycles. In the example here, with the input equal to '0.100', the output signal remains '1' until exactly four '1's are generated at the output. When the 4th '1' is returned to the DEC unit (as enable signal), the output from the DEC unit becomes '0', which also makes the output of the OR gate '0'.

TABLE I

A STEP-BY-STEP EXAMPLE OF BIT-STREAM GENERATION USING THE PROPOSED SERIAL DESIGN (INPUT VALUE=100, $m = 3$)

clk	DEC			m_or (Output Bit)
	in	out	en	
0	100	100	0	1
1	100	011	1	1
2	011	010	1	1
3	010	001	1	1
4	001	000	1	0
5	000	000	0	0
6	000	000	0	0
7	000	000	0	0

B. Parallel Design

Serial UC suffers from long latency as the number of processing cycles increases exponentially with the data precision. For a data precision equivalent to m binary bits, one needs to process unary bit-streams of 2^m bits in 2^m cycles. Given the relatively small area of unary logic compared to conventional binary designs, we can execute unary computations in parallel to reduce latency. To this end, we propose two designs for parallel generation of unary bit-streams: 1) an *approximate* design and 2) an *accurate* design. The approximate design offers lower hardware cost at the expense of accuracy, while the accurate design eliminates errors at a higher hardware cost. We define the degree of parallelism as p where $1 < p \leq m$.

1) *Approximate Design*: In the first design, the bit-stream generator is similar to the circuit shown in Fig. 4(a) but with some small modifications. In this design, the number of output ports and the value by which the DEC unit subtracts the input are both equal to the degree of parallelism or p . For example, for $p = 2$, the DEC unit subtracts the input by '0.010' or '0.000', depending on the enable input. Fig. 4(b) shows the proposed design, and Table II demonstrates a step-by-step example. We denote the output ports with $o_1, \dots, o_p$. Assume that the input binary value is '0.100' and $m = 3$. In this case, the DEC unit subtracts the input value by '0.000' in the first cycle. The output of subtraction is '0.100', which goes back to the input and OR gate. The m_or signal becomes '1' and drives all the outputs ($o_1, \dots, o_p$). At the same time, a '1' is returned to the enable input of the DEC unit. Next, the input ('0.100') is subtracted by '0.010', which produces '0.010'. The m_or signal remains on '1'. The output stream becomes 1, 1, 1. The input ('0.010') is subtracted again by '0.010', which produces '0.000' this time. Now the m_or signal becomes '0', which results in '0' for all output ports (the output stream becomes 0, 0, 0). Similar to the serial bit-stream generator, the FSM state changes, and the remaining output bits become '0'.

We note that if the number of '1's in the output ports is a multiple of p (e.g., four '1's in the example), this method produces an accurate output bit-stream. However, if the above condition is not met, there is an error in the generated bit-streams. For example, if the input value is '0.101', p is 2, and m is 3, the generated bit-stream contains four '1's, which is one bit of '1' short compared to the desired output bit-stream with five '1's. Table III reports the accuracy of the first parallel design in terms of mean absolute error (MAE) for converting data to unary bit-streams for different values of m and p .

2) *Accurate Design*: In the second method, we make some modifications to the first parallel design to accurately generate sub-bit-streams. Fig. 4(c) shows the proposed design, and Table II a step-by-step example of this method. This design includes a comparator, a decoder, p 2-to-1 MUXs, and a controller. In the first cycle, the select

TABLE II

A STEP-BY-STEP EXAMPLE OF BIT-STREAM GENERATION BY OUR PROPOSED PARALLEL DESIGNS (INPUT VALUE=100, $m = 3$ AND $p = 2$)

clk	Approximate design						Accurate design									CS
	DEC			m_or	o_1	o_2	DEC			m_or	rem_1	rem_2	o_1	o_2		
	in	out	en				in	out	en							
0	100	100	0	1	1	1	100	100	0	1	-	-	1	1	0	
1	100	010	1	1	1	1	100	010	1	1	-	-	1	1	0	
2	010	000	1	0	0	0	010	000	1	0	0	0	0	0	1	
3	000	000	0	0	0	0	000	000	0	0	0	0	0	0	1	

TABLE III

MAE (%) OF THE FIRST PARALLEL GENERATOR FOR DIFFERENT DEGREES OF PARALLELISM (2×, 4×, 8×)

Input Precision	First Parallel Generator		
	2×	4×	8×
$m = 5$	1.56	4.68	10.93
$m = 6$	0.78	2.34	5.46
$m = 7$	0.39	1.17	2.73
$m = 8$	0.19	0.58	1.36
$m = 9$	0.09	0.29	0.68
$m = 10$	0.04	0.14	0.34

input (sel) of the MUXs is '0'. In each clock cycle, the comparator compares the output of the DEC unit with p . When comparator output (CS) is '1', it means that there is a remainder less than p , and we need to change the controller's state (shown in Fig. 5) from *Normal* to *RemOut*. In the *RemOut* state, the controller sets $en = 0$ and $sel = 1$. When the sel signal becomes one, the output bit-streams will be fed with the output of the decoder ($rem_1 \dots rem_p$). The output of the decoder shows how many output bits must be set to '1'. For example, with $p = 4$, if the remainder is '0.011', we need to set o_1, o_2 , and o_3 to '1' by encoding the remainder value ($rem_1 \dots rem_4$) to '1110'. This parallel design converts the input value to p sub-bit-streams accurately in parallel.

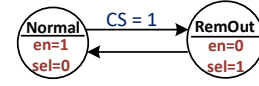


Fig. 5. The controller for accurate parallel design.

III. HARDWARE COST

In this section, we compare the hardware costs of the proposed serial and parallel unary bit-stream generators with that of the conventional comparator-based unary bit-stream generator commonly used in UC systems.

A. Serial Design

We developed an RTL VHDL description for the proposed comparison-free and the conventional comparator-based unary bit-stream generator. We synthesized the designs using the Synopsys Design Compiler v2018.06 with a 45nm FreePDK library [17] and also with Xilinx Vivado v2018.2 on a Virtex-7 FPGA. We report the synthesis results for different data bit-widths (i.e., $m = 4, 5, 6, 7$, and 8) and two cases of converting *one input* and *eight inputs* to unary bit-streams. When converting multiple inputs, the counter can be shared in the conventional design between several bit-stream generators to save the hardware cost. Table IV reports the synthesis results in terms of hardware area, power consumption, critical path latency (CP), and the number of occupied lookup tables (LUTs). As can be seen, the proposed (serial) generator achieves lower values

TABLE IV
SYNTHESIS RESULTS OF THE IMPLEMENTED UNARY BIT-STREAM GENERATORS (PROPOSED SERIAL VS. CONVENTIONAL COMPARATOR-BASED) FOR CONVERTING DIFFERENT NUMBERS OF INPUTS (N) AND DATA PRECISIONS (m).

N	m	Area (μm^2)		Power @1GHz (μW)		CP (ns)		Occupied LUT	
		Proposed	Conventional	Proposed	Conventional	Proposed	Conventional	Proposed	Conventional
1	4	19.2	114	17.9	167	0.21	0.46	1	4
	5	24.4	142	17.6	195	0.24	0.48	2	5
	6	25.3	169	18.5	218	0.27	0.52	2	8
	7	27.6	210	19.6	246	0.31	0.56	3	9
	8	35.3	237	20.7	267	0.38	0.61	3	10
8	4	153	265	140	231	0.21	0.46	8	14
	5	195	405	141	290	0.24	0.48	16	16
	6	202	530	148	331	0.27	0.52	16	24
	7	221	589	162	359	0.31	0.56	24	37
	8	282	760	165	394	0.38	0.61	24	43

TABLE V
HARDWARE AREA (μm^2), CRITICAL PATH LATENCY (ns), AND AREA DELAY PRODUCTS ($A \times D$) OF THE COMPARATOR-BASED, THE PROPOSED APPROXIMATE AND ACCURATE DESIGN FOR THE PARALLEL GENERATION OF UNARY BIT-STREAMS FOR 8-BIT PRECISION DATA.

Parallel Design	1×			2×			4×			8×		
	Area	CP	$A \times D$	Area	CP	$A \times D$	Area	CP	$A \times D$	Area	CP	$A \times D$
Comparator-based	237	0.61	144	451	0.61	275	901	0.61	550	1414	0.61	862
Approximate design	35	0.38	13.4	41	0.38	15	43	0.38	16	47	0.38	17
Exact design	35	0.38	13.4	52	0.40	21	70	0.41	28	128	0.45	57

than the conventional approach in all metrics. For instance, in the case of converting one input when $m = 8$, the circuit area of the proposed design is $35.3 \mu m^2$ compared to $237 \mu m^2$ of the comparator-based generator. This shows 85% saving in area. For the same case, the power consumption of the proposed design is $20.7 \mu W$, providing 92% saving compared to $267 \mu W$ of the prior design. In the case of converting eight inputs when $m = 8$, the circuit area of the proposed design is $282 \mu m^2$ compared to $760 \mu m^2$ of the conventional generator, which shows 62% saving in area. For this case, the power consumption of the proposed approach is $165 \mu W$, 58% lower compared to $394 \mu W$ of the comparator-based generator.

When increasing the number of inputs from one to eight, the proposed design still saves the hardware area and power cost considerably compared to the comparator-based generator. A higher saving rate (i.e., 85%) was observed for the case of converting a single input because the dominant cost in the conventional generator was the cost of the counter. However, when converting multiple inputs, the counter is shared between several generators, and the dominant is the cost of a new binary comparator for each additional input. Considering the lower cost of the proposed m -bit design compared to that of an m -bit binary comparator ($35 \mu m^2$ for the proposed vs. $73 \mu m^2$ for the comparator when $m=8$), the proposed design shows scalability, and similar cost savings are expected when increasing the number of inputs. As it can be seen in Table IV, in terms of CP, the proposed design provides a lower latency, which means for the same number of processing cycles it works faster and reduces the total latency. For example, for $m = 4$, our design reduces CP by 54% from $0.46 ns$ to $0.21 ns$, and for $m = 8$ by 38% from $0.61 ns$ to $0.38 ns$.

B. Parallel Design

Table V reports the hardware area, CP, and area-delay products of the conventional comparator-based, and the proposed approximate and accurate parallel designs for $2\times$, $4\times$, and $8\times$ degrees of parallelism. We report the hardware costs for processing 8-bit precision

data. We also report the cost of non-parallel ($1\times$) unary bit-stream generation in this table. As reported, for the case of $2\times$ parallel design, the hardware area costs of the proposed approximate and accurate parallel designs are $41.3 \mu m^2$ and $52.5 \mu m^2$, respectively, while the comparator-based design (Fig. 2(b)) occupies $450 \mu m^2$. The area-delay product of the approximate and accurate parallel designs are 15 and 21, respectively, while for the comparator-based design it is 275, $13\times$ greater than the accurate parallel design. In the case of $4\times$ parallel design, the area-delay product of the accurate design is 28, while for the comparator-based unary generator, it is 550. Finally, for $8\times$ parallel design, the area-delay product of the comparator-based design is $15\times$ and $50\times$ more than that of the accurate and approximate parallel design, respectively. Evidently, the proposed design significantly reduces the area, latency, and area-delay product of parallel unary bit-stream generation compared to the conventional comparator-based unary bit-stream generator.

IV. CASE STUDY

Next, we evaluate the hardware efficiency of the proposed bit-stream generator for four state-of-the-art UC systems for 1) Sorting, 2) Median filtering, 3) Robert's edge detection, and 4) Gamma correction.

A. Sorting Networks

Sorting is an important task in different applications from databases to artificial intelligence, scientific computing, and image processing. For applications that require high performance, sorting is often performed in hardware. A common approach for hardware sorting is to wire up a network of compare-and-swap (CAS) units. The incoming data is sorted as it passes the network. Each CAS block compares two input values and swaps the values at the output if required. There are two variants of CAS blocks: an "ascending ($\uparrow$)" type and a "descending ($\downarrow$)" type. Fig. 6 shows the CAS network for sorting eight input data.

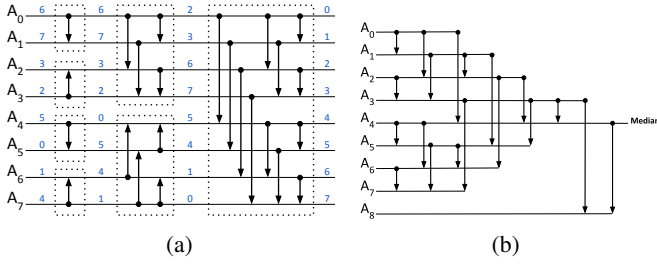


Fig. 6. The CAS network for (a) sorting 8 input data, and (b) a 3x3 median filter circuit [5].

The authors in [5] proposed an area- and power-efficient implementation for sorting networks based on UC. In their design, each CAS unit is implemented using one AND and one OR gate. Their design uses the conventional comparator-based design for converting data from binary to unary representation. To improve the sorting designs of [5], we replace the conventional unary bit-stream generator used in their designs with our proposed design. We compare the hardware cost for three cases of 1) the conventional implementation with weighted binary data (WB), 2) UC design with comparator-based unary bit-stream generator [5], and 3) UC design with the proposed converter. Table VI compares the hardware area and power costs in 45nm technology. The number of input data (i.e., sorting network size) ranges from 8 to 256 with data-width $m = 8$. As it can be seen, in the case of implementing 8-input sorting network (shown in Fig. 6), the sorting design with the proposed bit-stream generator occupies $2,242 \mu m^2$, while the design with comparator-based unary generator, and the weighted binary implementation consume $2,706 \mu m^2$ and $3,086 \mu m^2$ area, respectively. A similar saving is observed for the power consumption. The area and power savings with the UC designs increase as the sorting network size increases. Our proposed converter reduces the area and power consumption of the sorting circuits up to 84% and 93% compared to the conventional weighted binary and 17% and 36% compared to the comparator-based UC sorting.

We also implemented the parallel unary design of the sorting networks with the comparator-based and also with the proposed accurate parallel design with degrees of $2\times$, $4\times$, and $8\times$. Table VII reports the hardware area costs. We only compare the area footprint in this case as the power consumption is correlated with the area and a similar improvement is expected. The synthesis results show the superiority of the sorting circuit implemented with the proposed parallel generator over the sorting design with parallel comparator-based unary generator. For example, in the case of sorting 256 input data with the $8\times$ parallel design, the design with our accurate converter occupies $164,116 \mu m^2$ while the one with comparator-based generator consumes $396,594 \mu m^2$ area. This means 58.6% area saving with the proposed parallel unary generator.

B. Median Filtering

A median filter is a popular nonlinear filter widely used in image, speech, and signal processing applications [18], [19], [20]. It replaces each input data with the median of all the data in a local neighborhood. This results in filtering out impulse noise and smoothing the image while preserving important properties such as edge information. Sorting network-based median filter consisting of a network of CAS blocks is a common approach for the hardware design of median filter. The authors in [5] proposed a low-cost UC design for median filtering using CAS networks. We implemented and synthesized a 3×3 UC-based median filter design with both the

TABLE VI
SYNTHESIS RESULTS OF SORTING CIRCUITS WITH DIFFERENT NUMBERS OF INPUTS WITH CONVENTIONAL WEIGHTED BINARY (WB), COMPARATOR-BASED UC [5], AND PROPOSED DESIGN FOR $m = 8$.

Input	Area (μm^2)			Power @ 50MHz (μW)		
	WB	[5]	Proposed	WB	[5]	Proposed
8	3,086	2,706	2,242	120	130	83
16	10,534	5,444	4,680	490	250	170
32	32,508	11,196	9,833	1,750	490	360
64	90,691	23,279	20,719	5,480	960	750
128	242,049	49,756	44,799	15,480	1,800	1450
256	586,456	106,037	96,288	42,200	3,640	2960

TABLE VII
HARDWARE AREA COST OF UC SORTING CIRCUITS WITH DIFFERENT NUMBERS OF INPUTS FOR $2\times$, $4\times$, AND $8\times$ PARALLEL BIT-STREAM GENERATION WITH THE COMPARATOR-BASED [5] AND WITH OUR ACCURATE PARALLEL UNARY GENERATOR FOR $m = 8$.

Input	$2\times$		$4\times$		$8\times$	
	[5]	Proposed	[5]	Proposed	[5]	Proposed
8	2,266	1,193	4,021	1,597	7,531	2,059
16	4,668	2,780	8,171	3,983	15,321	4,907
32	9,951	6,504	17,572	9,855	33,102	11,702
64	21,678	15,113	38,696	23,920	73,307	27,616
128	49,750	36,948	90,179	61,286	172,190	68,678
256	112,551	87,277	206,461	149,334	396,584	164,116

comparator-based and the proposed unary generator. The hardware area costs are reported in Table IX. As it can be seen, our approximate and accurate parallel designs reduce the area cost of the state-of-the-art UC median filter [5] up to 78% and 69%, respectively. We further evaluate the performance of the UC design with the proposed parallel designs for different bit-stream lengths for 1000 sets of random input values. Table VIII reports the MAE results. As can be seen, the accurate designs achieve 0.00% MAE with a bit-stream length of 256. For the approximate parallel designs, increasing the degree of parallelism increases the MAE for a fixed bit-stream length. However, by increasing the bit-stream length, the error rate decreases quickly.

C. Robert's cross edge detection

We further implement Robert's cross edge detector circuit as proposed in [21]. Each Robert's cross operator consists of a pair of 2×2 convolution kernels that process an image pixel based on its three neighbors as follows:

$$S_{i,j} = 0.5 \times (|r_{i,j} - r_{i+1,j+1}| + |r_{i,j+1} - r_{i+1,j}|)$$

where $r_{i,j}$ is the value of the pixel at location (i,j) of the input image and $S_{i,j}$ is the output value computed for the same location in the output image. The circuit is shown in Fig. 7(a). In this design, two XOR gates are used to implement the absolute value subtractions and one MUX unit is used for the scaled addition. The four inputs to the XOR gates must be correlated bit-streams, but they should all be uncorrelated to the select input of the MUX, which is a bit-stream representing 0.5. For converting the four main inputs from binary to unary, we use the proposed parallel design. We evaluated the performance of both approximate and accurate parallel designs when used for converting the inputs of Robert's cross circuit. Table VIII reports the accuracy results for different bit-stream lengths and degrees of parallelism and Table IX compares the hardware area costs. As can be seen in the reported numbers, the proposed designs achieve a high accuracy for most cases. Especially with bit-stream

TABLE VIII

MAES OF THREE APPLICATION CASE STUDIES FOR DIFFERENT DEGREES OF PARALLELISM ($1\times$, $2\times$, $4\times$, $8\times$) AND BIT-STREAM LENGTHS FOR THE PROPOSED PARALLEL DESIGNS. THE INPUT DATA BIT-WIDTH (m) IS 8.

Design	Bit-stream Length	$1\times$		$2\times$		$4\times$		$8\times$	
		Accurate	Approximate	Accurate	Approximate	Accurate	Approximate	Accurate	Approximate
3×3 Median Filter	8	1.84	1.78	1.81	4.19	1.82	12.81	1.78	43.79
	16	1.02	0.99	0.95	2.13	1.01	5.46	1.00	14.14
	32	0.59	0.58	0.59	1.12	0.60	2.70	0.60	6.21
	64	0.39	0.39	0.38	0.63	0.39	1.37	0.39	3.33
	128	0.29	0.29	0.30	0.39	0.30	0.70	0.31	1.52
	256	0.00	0.00	0.00	0.30	0.00	0.88	0.00	2.04
	512	0.00	0.00	0.00	0.00	0.00	0.29	0.00	0.90
Robert's Cross Edge Detection	8	4.64	4.00	4.26	4.87	4.23	11.34	4.39	16.81
	16	2.16	2.08	2.11	2.62	2.15	5.95	2.00	12.33
	32	1.03	0.99	1.03	1.29	1.03	3.11	1.09	6.52
	64	0.54	0.54	0.55	0.69	0.54	1.56	0.53	3.37
	128	0.28	0.27	0.25	0.38	0.26	0.85	0.26	1.82
	256	0.11	0.10	0.12	0.13	0.10	0.39	0.11	0.84
	512	0.00	0.00	0.00	0.00	0.00	0.12	0.00	0.38
Gamma Correction	8	11.09	11.81	11.04	11.36	11.22	16.22	11.19	30.06
	16	7.74	7.64	7.95	8.05	8.38	8.89	8.10	15.64
	32	4.84	4.91	5.09	5.27	5.20	7.57	5.30	11.21
	64	3.52	3.43	3.55	3.63	3.51	3.96	3.69	6.97
	128	2.53	2.49	2.54	2.64	2.48	2.52	2.40	3.08
	256	1.76	1.72	1.75	1.85	1.74	1.86	1.72	1.89
	512	1.28	1.29	1.27	1.35	1.24	1.29	1.31	1.34

TABLE IX

HARDWARE AREA COST OF UC CIRCUITS FOR $2\times$, $4\times$, AND $8\times$ PARALLEL BIT-STREAM GENERATION WITH COMPARATOR-BASED (CB), AND WITH PROPOSED PARALLEL DESIGNS FOR $m = 8$

Design	$2\times$			$4\times$			$8\times$		
	CB	Approx.	Accurate	CB	Approx.	Accurate	CB	Approx.	Accurate
Median	2,349	1,104	1,205	4,131	1,280	1,524	7,695	1,632	2,359
Robert	1,175	439	484	2,098	466	574	3,943	520	843
Gamma	2,138	995	1,085	3,772	1,165	1,381	7,040	1,505	2,151

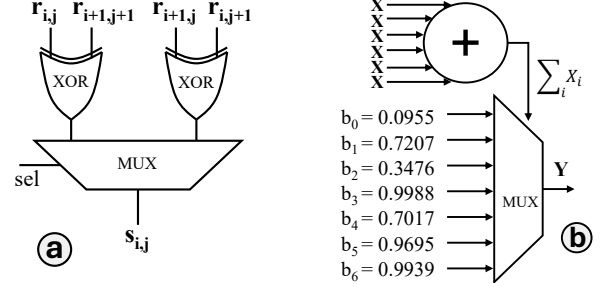


Fig. 7. UC designs for (a) Robert's cross edge detection [21], (b) Gamma correction function [15].

lengths of more than 32 bits, they achieve a MAE of less than 1%. The synthesis results show up to 86% area savings with proposed design compared the conventional comparator-based design.

D. Gamma Correction

Finally, we implement the Gamma correction circuit proposed in [15] and shown in Fig. 7(b). This circuit implements the Gamma correction function ($f(x) = x^{0.45}$) by approximating the function using a Bernstein polynomial and with a MUX-based architecture, called ReSC [15]. The inputs to this circuit are six independent bit-streams, each with a probability corresponding to the value x and seven correlated unary bit-streams, set to constant values corresponding to the Bernstein coefficients. We implemented the design with both comparator-based and proposed unary converters. Table IX compares the area cost numbers. We did not consider the cost of generating independent bit-streams in our area comparison as it's the same in both UC design approaches. As can be seen, the proposed design reduces the hardware area cost of the UC Gamma correction circuit by up to 78%. Table VIII further reports the accuracy results with both accurate and approximate parallel design. A higher error rate compared to the Robert and Median designs is due to the approximation of the Gamma correction function with the ReSC architecture.

V. CONCLUSION

In this work, we proposed a novel comparison-free unary bit-stream generation approach for the cost-efficient conversion of data from binary to unary bit-streams. We proposed a serial and two parallel (an *accurate* and an *approximate*) unary bit-stream generators. We evaluated the hardware cost of the proposed designs compared to the conventional comparator-based binary-to-unary converter. We further evaluated the accuracy performance for different bit-stream lengths and degrees of parallelism. Synthesis results showed up to 85% and 92% savings in the area and power costs, respectively, compared to the state-of-the-art comparator-based unary bit-stream generator.

The MATLAB and synthesizable HDL source codes associated with this work are publicly available at: <https://github.com/faeze-bnt/Bit-stream-Gen>.

VI. ACKNOWLEDGEMENT

This work is supported in part by the National Science Foundation under grants #2019511, #2339701, and a generous gift from NVIDIA.

REFERENCES

- [1] S. S. Gill, H. Wu, P. Patros, C. Ottaviani, P. Arora, V. C. Pujol, D. Haunschild, A. K. Parlikad, O. Cetinkaya, H. Lutfiyya *et al.*, “Modern computing: Vision and challenges,” *Telematics and Informatics Reports*, p. 100116, 2024.
- [2] D. Wu, J. Li, R. Yin, H. Hsiao, Y. Kim, and J. San Miguel, “Ugemm: Unary computing architecture for gemm applications,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 377–390.
- [3] A. Alaghi, W. Qian, and J. P. Hayes, “The promise and challenge of stochastic computing,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 8, pp. 1515–1531, 2017.
- [4] W. J. Poppelbaum, A. Dollas, J. Glickman, and C. O’Toole, “Unary processing,” in *Advances in computers*. Elsevier, 1987, vol. 26, pp. 47–92.
- [5] M. H. Najafi, D. J. Lilja, M. D. Riedel, and K. Bazargan, “Low-cost sorting network circuits using unary processing,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 26, no. 8, pp. 1471–1480, 2018.
- [6] S. R. Faraji and K. Bazargan, “Hybrid binary-unary hardware accelerator,” in *Proceedings of the 24th Asia and South Pacific Design Automation Conference*, 2019, pp. 210–215.
- [7] M. H. Najafi, D. Jenson, D. J. Lilja, and M. D. Riedel, “Performing stochastic computation deterministically,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 12, pp. 2925–2938, 2019.
- [8] A. H. Jalilvand, M. H. Najafi, and M. Fazeli, “Fuzzy-logic using unary bit-stream processing,” in *2020 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2020, pp. 1–5.
- [9] M. Shoushtari Moghadam, S. Aygun, F. S. Banitaba, and M. H. Najafi, “All you need is unary: End-to-end unary bit-stream processing in hyperdimensional computing,” in *Proceedings of the 29th ACM/IEEE International Symposium on Low Power Electronics and Design*, 2024, pp. 1–6.
- [10] A. Khataei, G. Singh, and K. Bazargan, “Approximate hybrid binary-unary computing with applications in bert language model and image processing,” in *Proceedings of the 2023 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2023, pp. 165–175.
- [11] D. Wu, J. Li, Z. Pan, Y. Kim, and J. S. Miguel, “ubrain: A unary brain computer interface,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 468–481.
- [12] A. H. Jalilvand, F. S. Banitaba, S. N. Estiri, S. Aygun, and M. H. Najafi, “Sorting it out in hardware: A state-of-the-art survey,” *ACM Transactions on Design Automation of Electronic Systems*, 2025.
- [13] C. Ma, Y. Sun, W. Qian, Z. Meng, R. Yang, and L. Jiang, “Go unary: A novel synapse coding and mapping scheme for reliable rram-based neuromorphic computing,” in *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2020, pp. 1432–1437.
- [14] J. Smith, “Space-time algebra: A model for neocortical computation,” in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 289–300.
- [15] W. Qian, X. Li, M. D. Riedel, K. Bazargan, and D. J. Lilja, “An architecture for fault-tolerant computation with stochastic logic,” *IEEE transactions on computers*, vol. 60, no. 1, pp. 93–105, 2010.
- [16] S. Asadi, M. H. Najafi, and M. Imani, “A low-cost fsm-based bit-stream generator for low-discrepancy stochastic computing,” in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2021, pp. 908–913.
- [17] “NCSU FreePDK 45nm Lib.” <https://eda.ncsu.edu/freepdk/freepdk45/>.
- [18] D. S. Richards, “Vlsi median filters,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 38, no. 1, pp. 145–153, 1990.
- [19] G. George, R. M. Oommen, S. Shelly, S. S. Philipose, and A. M. Varghese, “A survey on various median filtering techniques for removal of impulse noise from digital image,” in *2018 Conference on Emerging Devices and Smart Systems (ICEDSS)*. IEEE, 2018, pp. 235–238.
- [20] L. Yin, R. Yang, M. Gabbouj, and Y. Neuvo, “Weighted median filters: a tutorial,” *IEEE Transactions on circuits and systems II: analog and digital signal processing*, vol. 43, no. 3, pp. 157–192, 1996.
- [21] A. Alaghi, C. Li, and J. P. Hayes, “Stochastic circuits for real-time image-processing applications,” in *Proceedings of the 50th Annual Design Automation Conference*, 2013, pp. 1–6.